Project 10.1 — FitPal-Persistent-Fitness-Coach-With-Cross-Session-Memory

A practice twin of Project 10 — MemoryKeeper. Same skills — two memory systems on a LangGraph graph (a per-conversation checkpointer and a per-user long-term store), a node that *recalls* you, a node that *decides what's worth remembering*, and a real `forget_me`. A **new, real-world job**. **There are no steps here on purpose.** Read the story, build it yourself, and only open the hints or the solution if you get stuck.

Type: Single-agent chat assistant (LangGraph: checkpointer + long-term store) · **Stack:** Python, LangGraph, `langchain-openai`, `SqliteSaver`, `InMemoryStore`, Pydantic · **Level:** Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

A good fitness coach remembers you. You tell them once that you're training for a 10k, that you only have dumbbells at home, that your left knee is bad so no deep squats — and they don't make you repeat it every session. A chatbot coach that forgets all of that between conversations is worse than useless: it'll happily program the one exercise your knee can't take.

You are going to build **FitPal**: a coaching assistant that remembers the things that matter across *separate* conversations — and, because this is health information, lets you wipe it completely whenever you want.

It needs **two** kinds of memory, and keeping them straight is the whole design:

- **Short-term** — the messages of *this* session, so it follows the conversation. LangGraph's **checkpointer** (keyed by `thread_id`) handles that.
- **Long-term** — durable facts about *you*: your goal, your equipment, your injuries, your schedule. These must outlive any one conversation, so they live in a **Store** keyed by `user_id`, which a brand-new session reads before it answers.

The hard part isn't reading those facts back — it's **deciding what's worth saving**. "I went for a run today" is chatter. "I have a bad left knee, avoid deep squats" is a durable fact that should shape every future plan. FitPal decides that for itself, saves durable facts under stable keys (so "actually my goal is now a half-marathon" *updates* the goal instead of leaving two), and offers a `forget_me` that deletes **everything** — the privacy promise that any health assistant has to keep.

The one lesson of this project: an assistant that remembers you needs two memory systems kept separate (this conversation vs. facts about you), the judgement to keep only durable facts, and a delete that truly forgets. Get those right and a new session already knows your knee is bad before it writes a single workout.

How it works, from a message to a remembered plan (picture the whole machine)

Read this once and the whole project should form in your head.

Everything runs on a small LangGraph graph: `recall → chat → write_back`. When you send a message, **recall** reads your long-term facts from the Store (keyed by your `user_id`) and puts them where the chat can see them. **chat** answers with those facts folded into its system prompt — so it knows your goal, your gear, and your bad knee without you repeating them, even in a conversation it has never seen. **write_back** then looks at what you just said and decides, with a structured yes/no, whether it holds a durable fact worth keeping: a goal, equipment, an injury, a standing preference — yes; "felt tired today" — no. If yes, it saves it under a **stable key**, so telling it a new goal *updates* the old one instead of stacking a contradiction.

The two memories never mix. The **checkpointer** (`SqliteSaver`, keyed by `thread_id`) holds *this* conversation's messages and survives a restart. The **Store** (keyed by `user_id`) holds the durable facts, so a brand-new `thread_id` still recalls you. And because it's health data, `forget_me` lists every stored item for you and deletes each one — after that, a fresh session genuinely knows nothing about you.

That's the whole machine: **recall your facts → coach with them in context → decide what new durable fact (if any) to save, under a stable key → keep two memories separate → forget everything on request.**

What you are building (the pieces)

- A LangGraph graph `recall → chat → write_back`, compiled with a `SqliteSaver` **checkpointer** and an `InMemoryStore`.
- A **Store layer**: `recall_facts`, `save_fact` (same key = update), `forget_user` (delete all).
- A **write-back decision** (`MemoryDecision` via structured output): save durable facts (goal, equipment, injury, preference), skip chatter.
- A **chat node** that folds recalled facts into its coaching system prompt.
- A **`forget_me`** command that wipes everything for the user.

Inputs: your messages. **Outputs:** coaching that remembers you across sessions; a clean forget. **Rules:** this-conversation vs. facts-about-you go in different memories; only durable facts are saved; a rename updates; `forget_me` removes everything.

A real example to build and test against

- Session 1: "I'm training for a 10k, I only have dumbbells, and my left knee is bad — no deep squats." → FitPal saves a `goal`, an `equipment`, and an `injury` fact.
- A brand-new session, same user: "give me a leg workout" → it already knows to avoid deep squats and to use dumbbells.
- "I felt a bit tired today." → **not** saved (chatter, not a durable fact).
- "Actually my goal is a half-marathon now." → **updates** the existing `goal`, not a second one.
- `forget_me` → deletes the goal, equipment, and injury; a fresh session knows nothing about you.

How to practice this

1. Read this story. **Don't open the solution yet.** (Read Doc09's long-term memory section if it's new.)
2. Plan it on paper: for each fact — goal, bad knee, "tired today", this session's last message — decide which memory it belongs in.
3. Build it yourself. A good order: a checkpointer-only chat (watch a new thread forget you) → add the Store + recall → add the write-back decision → add `forget_me`. Your call.
4. Test it on the real example: cross-session recall, an ignored chatter line, a goal update, and a clean forget.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

Any assistant people come back to — a coach, a tutor, a personal-finance helper, a health companion — is far more useful when it remembers you across sessions, and far more trustworthy when it can forget you on demand. The architecture is always the same two memories kept separate (this conversation vs. durable facts about you), the same hard judgement of what's worth keeping, and the same privacy-respecting delete. FitPal makes the stakes concrete: a coach that forgets your bad knee is dangerous, and health facts *must* be deletable. Build it once and you can give any assistant real, respectful memory.

Done when

- A brand-new session recalls your goal, equipment, and injury (the Store, keyed by `user_id`).
- A chatter line ("felt tired today") is **not** saved; a goal change **updates** the existing goal.
- `forget_me` removes every stored fact; a fresh session then knows nothing.
- You can say which memory any given fact belongs in — checkpointer or Store.

- Your tests pass: settings, the Store helpers (recall/save/update/forget, no model), and a live run (cross-session recall, write-back, forget).

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.